



tecnun

CAMPUS TECNOLÓGICO DE LA UNIVERSIDAD DE NAVARRA
NAFARROAKO UNIBERTSITATEKO CAMPUS TEKNOLOGIKOA
Escuela Superior de Ingenieros · Ingeniarien Goi Mailako Eskola

● C++ Laboratory Lessons

Lesson N° 4



Computer Programming II

Paul Bustamante / Jesús Paredes



OUTLINE

OUTLINE.....	1
1. Introduction.....	1
1.1 Exercise 1: Dynamic memory allocation for vectors.....	1
1.2 Exercise 2: Working with dynamic structures.....	2
1.3 Exercise 3: Dynamic over Dynamic.....	2
1.4 Exercise 4: Dynamic matrices.....	2
1.5 Exercise 5: Vector by Matrix.....	3

1. Introduction.

As we have been doing so far, the first exercise of this lesson you should done alone, so you can gain more experience in programming in C++. In some exercises, the code is given, but it does not mean you only have to write it on your computer; you should try to complete them on your own.

1.1 Exercise 1: Dynamic memory allocation for vectors

In this exercise, we will make use of the dynamic memory allocation to store a number of values (obtained at random, between 0 and 100) and sort from highest to lowest.

The code that gets the numbers and performs the dynamic memory allocation is given. You should try to implement the rest of the code. Create the project *Exer1* and the file *sort.cpp* to write the code.

```
// Sorts using dynamic memory
// Tecun - PBM @ 2024
#include <iostream>
using namespace std;
void main(void)
{
    int Num;           //Number of data
    int *data;         //pointer to int

    cout << "How many numbers do you want to introduce?:";
    cin >> Num;

    //dynamic memory allocation
    data = new int[Num];
    if (data == NULL) { cout << "Error"; return; }

    //Fill the vector
    for (int i=0;i<Num;i++){
        cout << "Give data[" << i << "]:?";
        cin >> data[i];
    }
    //sort the data -> Write here the sorting algorithm
    //. . .

    //print the sorted data
    for (int i=0;i<Num;i++)
        cout << i << ":" << data[i] << endl;

    //free memory
    delete [] data;
}
```



1.2 Exercise 2: Working with dynamic structures.

In this exercise, you will work more in the use of structures in C++, using the dynamic memory allocation.

The program must ask for the number of element, in order to allocate the memory. See the program in the next lines:

```
//Paul Bustamante - Tecnun2024
#include <iostream>
using namespace std;
struct student
{
    char name[50];          //static array
    double mark;
};
void main()
{
    student *list;          //List of students
    int num=0,pos=0;
    char ch;

    cout << "Give me Num?:";    cin >> num;
    list = new student[num];    //dynamic array

    for (int i=0;i<num;i++){
        cout << "Give Name Mark?";
        cin >> list[i].name >> list[i].mark;
    }
    double min=list[0].mark;    //get the first
    //list all
    cout << "-----" << endl;
    for (int i=0;i<num;i++){
        cout << list[i].name << ", " << list[i].mark << endl;
        if (list[i].mark < min) {
            min=list[i].mark;
            pos=i;
        }
    }
    cout << "-----" << endl;
    cout << "worst case:" <<list[pos].name<<" with " << list[pos].mark
<<endl;
    delete [] list;          //deallocate the memory
}
```

1.3 Exercise 3: Dynamic over Dynamic

In order to develop this exercise, you must continue working with the exercise 2. In this case, you must change this element `char name[40];` in the student structure for the next one: `char *name;` //dynamic array

You must change all that you need in the program in order to do the same that in exercise 2.

1.4 Exercise 4: Dynamic matrices.

In this exercise, you will work on the allocation of *dynamic memory for matrices*, for which you must follow these steps:

- Create the pointer array of the following data type: `float **data;`
- Allocate memory for the pointer array: `data = new float*[row];`



- c. Write a loop to allocate memory for **col** columns of each row:
`for (int i=0;i<row;i++) data[i] = new float[col];`
- d. We have already created the matrix, now we can work with it with indexes, using the brackets [], as data[i][j].
- e. Finally, we make another loop to free the memory of each row and then we free the memory allocated to the array of pointers, in that strict order.

In the following example, you will see how to allocate memory for an array of **rowxcol**:

```
// File matrix.cpp
// Creating matrixes using dynamic memory
// Tecun - PBM @ 2024
#include <iostream>
using namespace std;
void main(void)
{
    int row; //number of rows
    int col; //number of columns
    float **data;
    cout << "Num. Rows:"; cin >> row;
    cout << "Num. Columns:"; cin >> col;
    data = new float*[row]; //rows vector
    //memory allocation for the columns of each row
    for (int i=0;i<row;i++)
        data[i] = new float[col];

    //now you can use the matrix
    for (int f=0;f<row;f++)
        for (int c=0;c<col;c++) data[f][c] = (float)rand()/RAND_MAX;

    //print the values
    for (int f=0;f<row;f++){
        for (int c=0;c<col;c++) {
            cout << f << "," << c << ":"<<data[f][c]<< " ";
        }
        cout <<endl;
    }
    //free memory
    for (int f=0;f<row;f++) delete [] data[f]; //free rows
    delete [] data; //free the vector
}
```

1.5 Exercise 5: Vector by Matrix.

In this exercise, you will work more on the allocation of *dynamic memory for vectors and matrices*, in order to practice more with them.

You must create a program to multiply a **vector by a matrix**. To reach this objective, you must ask to the user for the number of rows and columns of the matrix and the number of data for the vector.

Remember the mathematical rules about this issue: the number of data in the vector is equal to the number of rows in the matrix:

$$\begin{bmatrix} a & b & c \end{bmatrix} * \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} = [(a * 1 + b * 3 + c * 5) \quad (a * 2 + b * 4 + c * 6)]$$